

Estructuras como Parámetros de Funciones

Asignatura: Fundamentos de la Programación

Docente: Ing. Raúl Fernández Fajardo



Estructuras como Parámetros de Funciones

Asignatura: Fundamentos de la Programación

Docente: Ing. Raúl Fernández Fajardo

Objetivo General

Comprender cómo pasar estructuras como parámetros en funciones en C++, para organizar mejor el código, reutilizar lógica y facilitar la manipulación de datos estructurados.

Objetivos Específicos

- Explicar cómo se pasa una variable estructurada a una función en C++.
- Aplicar funciones que reciben estructuras como parámetros para mostrar, modificar o procesar información.

¿Por qué pasar estructuras a funciones?

En programación, es común tener funciones que necesitan usar varios datos relacionados (como nombre, edad, etc.).

Si esos datos ya están agrupados en una estructura (struct), es mucho más limpio y claro pasar la estructura completa a la función, en lugar de pasar cada dato por separado.

Esto permite:

- Código más limpio
- Reutilización de funciones
- Mejor organización
- Facilidad de mantenimiento

Formas de pasar una estructura

- **Por valor** (copia del contenido)
- **Por referencia** (trabajo directo sobre el original)

Formas de pasar una estructura

1. Por valor (copia del contenido)

Se pasa **una copia** de la estructura a la función.

Cualquier cambio dentro de la función **no afecta** al original.

```
void mostrarPersona(Persona p) {  
    cout << "Nombre: " << p.nombre << endl;  
}
```

 Ideal cuando solo se necesita leer la estructura.

2. Por referencia (trabajo directo sobre el original)

Se pasa la **referencia** (acceso directo) a la estructura, usando el símbolo **&**.

Los cambios dentro de la función **afectan** al original.

```
void cambiarEdad(Persona &p) {  
    p.edad = 99;  
}
```

 Útil cuando necesitas **modificar los valores originales**.

Nota: Recuerda que puedes probar el código con un compilador en línea.
<https://www.programiz.com/cpp-programming/online-compiler>

Ejemplo de estructuras (con valor – por referencia)

🧠 ¿QUÉ OCURRE EN EL EJEMPLO?

1. Se declara una estructura Persona con nombre y edad.
2. En main(), el usuario llena los datos.
3. La función mostrarPersona() se llama y **recibe una copia** de la estructura → solo imprime los datos.
4. Luego, se llama cambiarEdad() pasando la estructura por **referencia** → modifica directamente edad.
5. Al volver a mostrar la estructura, se ve el cambio reflejado.

El símbolo & en el parámetro Persona &p indica que la función **trabaja directamente sobre la variable original**, no sobre una copia.

🧠 ¿Qué significa esto?

Cuando llamas a cambiarEdad(persona1);, **la función modifica el valor de edad de persona1 directamente**, porque se pasó por referencia.

¿Y si NO se usa &?

Si no usas &, se pasa una copia.
Cambios dentro de la función no afectan al original.

Nota: Recuerda que puedes probar el código con un compilador en línea.
<https://www.programiz.com/cpp-programming/online-compiler>

Estructuras como Parámetros de Funciones

Asignatura: Fundamentos de la Programación

Docente: Ing. Raúl Fernández Fajardo

Estructuras como Parámetros de Funciones

Asignatura: Fundamentos de la Programación

Docente: Ing. Raúl Fernández Fajardo

